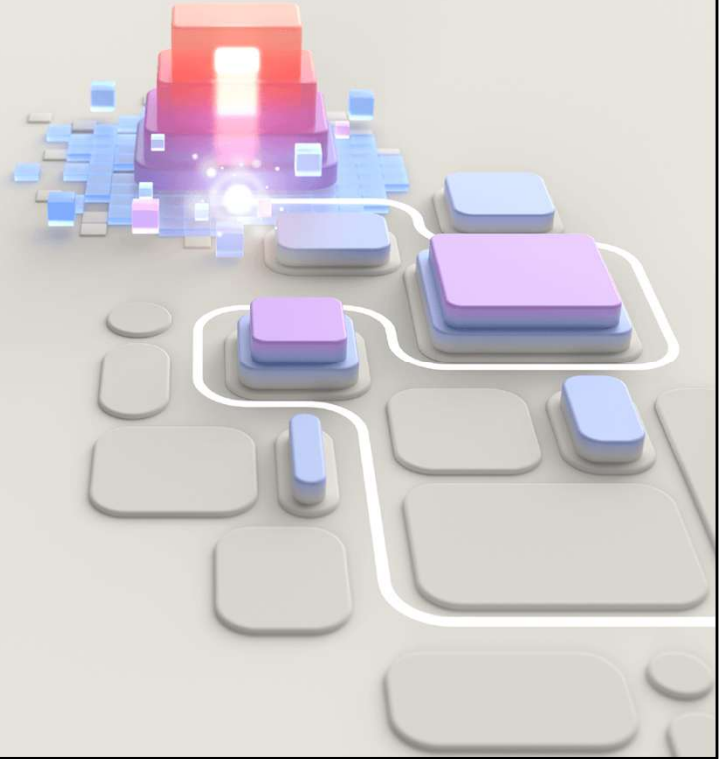




DP-300

Implement scalable database solutions using Azure SQL

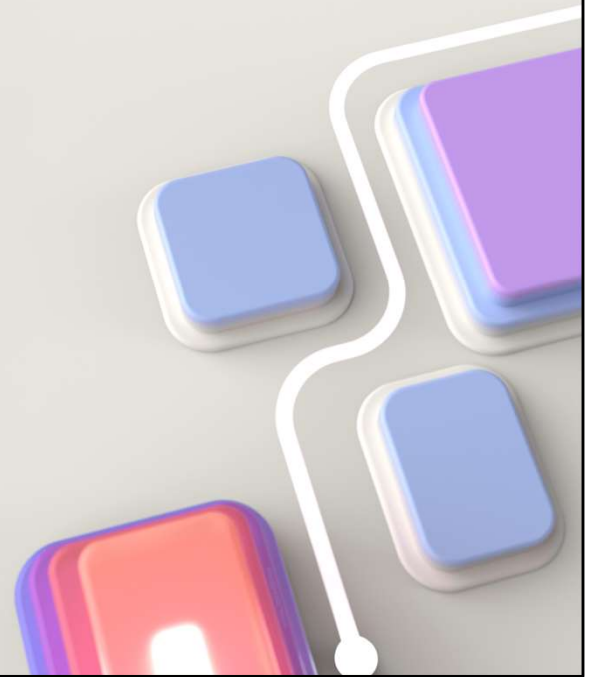


© Copyright Microsoft Corporation. All rights reserved.

Automate database tasks for Azure SQL

<https://learn.microsoft.com/training/paths/automate-tasks-sql-server/>

© Copyright Microsoft Corporation. All rights reserved.



<https://learn.microsoft.com/training/paths/automate-tasks-sql-server/>

Objectives

- Automate deployments
- Create Scheduled Tasks in SQL Server
- Create notifications for failures and performance alerts
- Configure automation for PaaS Services

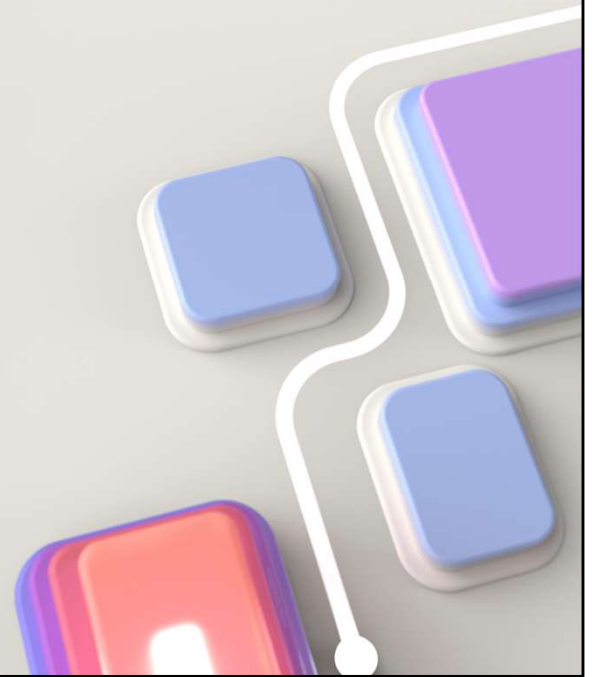
© Copyright Microsoft Corporation. All rights reserved.

A common goal for database administrators in many environments is to automate as many of their repetitive tasks. This can be as simple as using scripting to automate a backup process, and as complex as building a fully automated alerting system. This Learning Path provides details of automating tasks to simplify the DBA's job. Methods include scheduling tasks for regular maintenance jobs, as well as multi-instance administration and configuration of notifications for task success or failure or non-completion.

Automate database deployment

<https://learn.microsoft.com/training/modules/configure-automatic-deployment-azure-sql-database/>

© Copyright Microsoft Corporation. All rights reserved.



Objectives



How to automate deployment by using ARM templates and Bicep



How to automate deployment by using PowerShell and CLI



Monitor deployments

© Copyright Microsoft Corporation. All rights reserved.

Azure deployment methods

ARM templates and Bicep files

Enable the most comprehensive and customizable deployment model for Azure resources

Azure PowerShell

Commonly used for resource changes and status retrieval

Azure CLI

Like PowerShell, Azure CLI is designed for command-line interface operations, but it is built on the Bash shell. It's a versatile tool for managing Azure resources across different platforms.

Azure Portal

Acts as a graphical interface to Azure ARM and can be used to generate ARM templates

Azure DevOps

Deployments are carried out using Azure Pipelines. Azure Pipelines allows you to automate the build, testing, and deployment of your code

© Copyright Microsoft Corporation. All rights reserved.

PowerShell

PowerShell provides a core module known as Az, which has child resource providers for nearly all Azure services. For example, Az.Compute would cover Azure Virtual Machines. PowerShell is more commonly used for resource modification and status retrieval. While it is possible to create resources using PowerShell, it is not typically used for complex deployments. PowerShell can also be used to deploy ARM templates, so in a sense it supports both declarative and imperative models.

Azure CLI

The Azure Command Line Interface, or CLI, is similar to PowerShell in that it can be used either imperatively or declaratively. Much like PowerShell and ARM templates, the Azure CLI provides a mechanism to deploy or modify Azure Resources.

Azure Portal

The Azure Portal is a graphical interface to Azure ARM. Any resources you build and deploy using the Portal will have an ARM template that you can capture by clicking "Export Template" in the Settings

ARM Templates

ARM templates allow you to create and deploy an entire infrastructure in a declarative framework. For example, you can deploy not only a virtual machine, but its network and storage dependencies in one document. Resource manager also supports orchestration, which manages the deployment of interdependent resources so that they are created in the correct order. For example, a VM is dependent on the existence of a virtual network, so the framework will deploy the network (or check for the existence of the network) before attempting to build the VM. ARM templates also support extensibility, which allow you to run PowerShell or Bash scripts on your resources after they are deployed.

Bicep

Bicep isn't meant to be a general programming language. It's meant as a tool to allow you to create a file declaring Azure infrastructure resources and properties that can be used throughout the development lifecycle, allowing resource deployment in a consistent manner.

Azure Resource Manager (ARM) templates



An ARM template is a JavaScript Object Notation file that describes the Azure Resources to be deployed within it



ARM templates provide a declarative syntax to interact with the Azure API



ARM templates can be deployed through PowerShell, Azure CLI, and Azure DevOps pipelines



Microsoft offers pre-built templates for common deployment scenarios

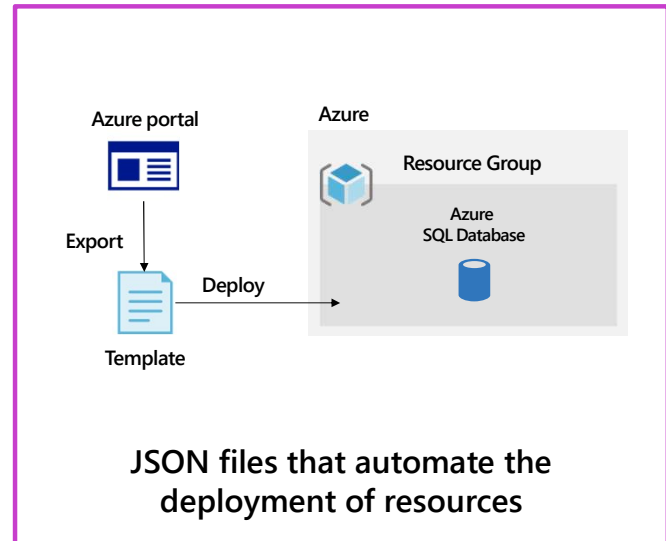
© Copyright Microsoft Corporation. All rights reserved.

Azure ARM Templates have the benefit of being able to deploy a full set of resources in one single declarative template. This includes the ability to build dependencies into the templates, as well as using parameters to change deployment values at deployment time. Once you have a template, you can deploy it several ways including using an Azure DevOps pipeline, or through the custom deployments blade in the Azure Portal. The benefit of these deployments is that they use a declarative model which defines what should be created and then the ARM framework determines how to deploy it. The alternative to the declarative model is the imperative model. Imperative frameworks include PowerShell and the Azure CLI which follow a prescriptive order of tasks to be executed.

In most cases the database administrator will not be writing their own ARM templates from scratch. You may either build them from the Azure Portal or using a template from the quick start templates that are provided by Microsoft on GitHub, for example <https://github.com/Azure/azure-quickstart-templates/tree/master/>

Azure Resource Manager (ARM) template benefits

-  Improves consistency
-  Repeatable and modular
-  Reduce error caused by manual mistakes
-  Promotes reuse
-  Simplifies orchestration, parallel deployment



© Copyright Microsoft Corporation. All rights reserved.

ARM templates offer the following benefits:

- **Repeatable** – ARM templates are *idempotent*, which means it allows you to repeatedly deploy your infrastructure throughout the development lifecycle and have confidence your resources are deployed in a consistent manner.
- **Orchestration** – ARM templates take care of the complexities of ordering operations for deployments and when possible will deploy resources in parallel rather than serial for faster deployments.
- **Modular** – ARM templates can be split and combined at will, so you can create the deployments you need.
- **Exportable code** – A great way to learn the template syntax is to export the current template. Exporting templates allow you to easily recreate your environment for Disaster Recovery or documentation purposes.
- **Authoring tools** – ARM templates can be authored using the freely available Visual Studio Code and the template tool extension. It provides intellisense, syntax highlighting, in-line help, and many other language functions. In addition to Visual Studio Code, you can also use Visual Studio.

ARM template



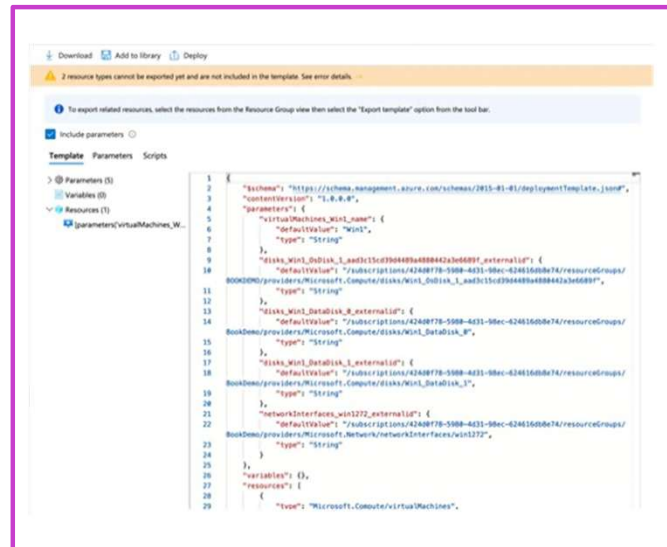
Deploys a full set of resources in one single declarative template



Dependencies and parameters can be built



Templates may be exported from the portal



© Copyright Microsoft Corporation. All rights reserved.

Azure ARM Templates have the benefit of being able to deploy a full set of resources in one single declarative template. This includes the ability to build dependencies into the templates, as well as using parameters to change deployment values at deployment time. Once you have a template, you can deploy it several ways including using an Azure DevOps pipeline, or through the custom deployments blade in the Azure Portal. The benefit of these deployments is that they use a declarative model which defines what should be created and then the ARM framework determines how to deploy it. The alternative to the declarative model is the imperative model. Imperative frameworks include PowerShell and the Azure CLI which follow a prescriptive order of tasks to be executed.

Creating an ARM template

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "",
  "apiProfile": "",
  "parameters": { },
  "variables": { },
  "functions": [ ],
  "resources": [ ],
  "outputs": { }
}
```

© Copyright Microsoft Corporation. All rights reserved.

An ARM template is a JSON (JavaScript Object Notation) document that describes the resources that will be deployed within an Azure Resource Group.

The schema file, which is referenced on the first line the above example describes the version of the template language. This file is supplied by Microsoft to define the Azure API. The content version number is only defined by you as needed and should align with your internal versioning standard. The API profile serves as a collection of API versions for this environment. This is especially useful when deploying resources to different environments like national clouds and commercial clouds, which may have differing resource providers. Parameters are values that are provided to the template in order to customize resources at deployment time, whereas variables are values that are used as JSON fragments to simplify template language expressions. You can also include user defined functions within the template. The resource component defines what resources are getting defined in the template. The resources are the resource types that you are deploying.

ARM template deployment

Azure PowerShell

```
New-AzResourceGroupDeployment -Name ExampleDeployment -ResourceGroupName  
ExampleResourceGroup -  
-TemplateFile c:\MyTemplates\azuredeploy.json -  
-TemplateParameterFile c:\MyTemplates\storage.parameters.json
```

Azure Command Line Interface (CLI)

```
az deployment group create --resource-group SampleRG --template-file `  
'\path\template.json'
```

© Copyright Microsoft Corporation. All rights reserved.

With PowerShell you have several options for the scope of your deployment. You can deploy to a resource group, a subscription, a Management Group (a collection of subscriptions under the same Azure template and commonly used in large enterprise deployments), or a tenant. ARM templates are parameterized, and you will need to pass in parameters, either inline or through the use of a parameter file as shown in the example below.

The Azure CLI allows the same options for deployment scope as you have with PowerShell

Demo: Exploring an Azure quick start template

**Review the quick start template for
provisioning an Azure SQL Database**

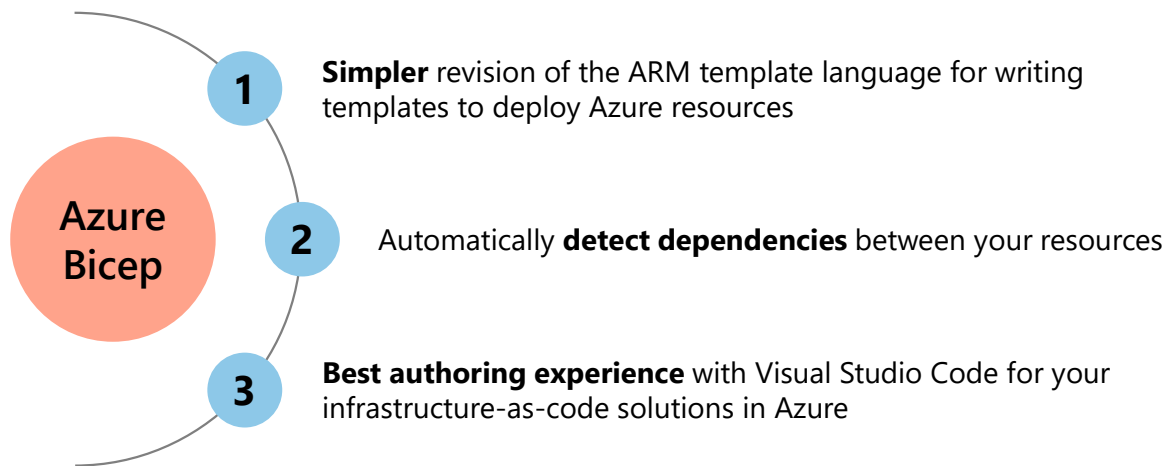
© Copyright Microsoft Corporation. All rights reserved.

In this demo use the following template:

<https://github.com/Azure/azure-quickstart-templates/tree/master/quickstarts/microsoft.sql/sql-database>

Explore the template properties and the options available.

What is Azure Bicep?



© Copyright Microsoft Corporation. All rights reserved.

Azure Bicep är Microsofts moderna språk för Infrastructure as Code i Azure. Det är en förenklad och mer lättläst version av ARM-template-språket, vilket gör det enklare att skriva och underhålla mallar som skapar resurser som VM:ar, nätverk, lagring och databaser. Bicep hjälper dig dessutom att automatiskt hantera beroenden mellan resurser, så att de skapas i rätt ordning utan att du behöver skriva mycket extra logik. En annan stor fördel är den mycket bra utvecklarupplevelsen i Visual Studio Code, med syntaxhjälp, IntelliSense och validering. Resultatet blir snabbare, säkrare och mer konsekventa Azure-deployments.

Ordlek → arm och biceps haha 😊

Azure Bicep is a declarative language that allows you to deploy Azure resources. Bicep provides a first-class authoring experience that is concise, reliable and allow for code reuse, and it's commonly described as an Infrastructure-as-Code (IaC) tool.

Bicep isn't meant to be a general programming language. It's meant as a tool to allow you to create a file declaring Azure infrastructure resources and properties that can be used throughout the development lifecycle, allowing resource deployment in a consistent manner.

Azure Bicep

Target your deployment to a resource group, subscription, management group, or tenant by using different options:

Deploy a local Bicep file: Use the `--template-file` parameter in the deployment command

Deploy a remote Bicep file: Not supported currently

Deploy template specs: Not supported currently by Azure CLI; but you can create a Bicep file with the `Microsoft.Resources/templateSpecs` resource to deploy

© Copyright Microsoft Corporation. All rights reserved.

Azure Bicep is a declarative language that allows you to deploy Azure resources. Bicep provides a first-class authoring experience that is concise, reliable and allow for code reuse, and it's commonly described as an Infrastructure-as-Code (IaC) tool.

Bicep isn't meant to be a general programming language. It's meant as a tool to allow you to create a file declaring Azure infrastructure resources and properties that can be used throughout the development lifecycle, allowing resource deployment in a consistent manner.

Azure Bicep benefits



Continuous full support



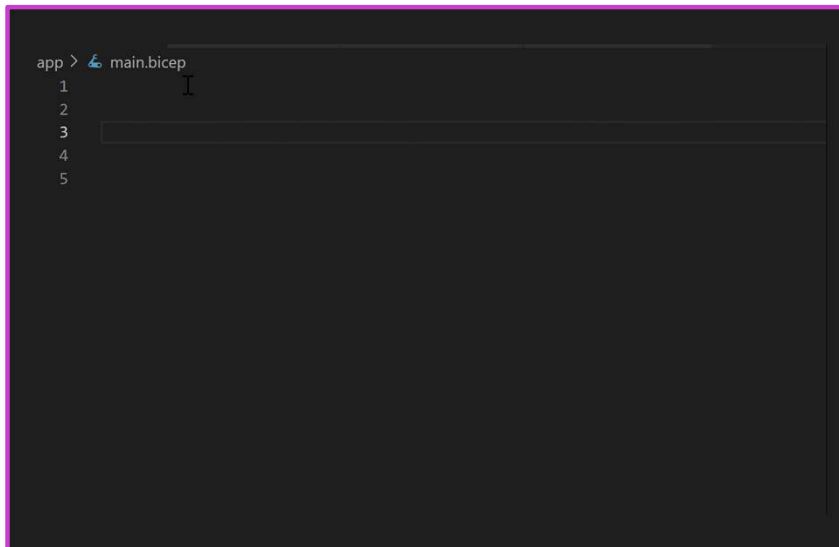
Simple syntax



Easy to use



Rich editor



© Copyright Microsoft Corporation. All rights reserved.

The following are some benefits of Bicep:

- **Continuous full support** – Bicep provides support for all resource types and API versions for Azure services, which means that as soon as a resource provider introduces new resource types and API versions, you can use them in your Bicep file without waiting for a tool update.
- **Simple syntax** – Compared to an equivalent JSON file, Bicep files will be more concise and easier to read.
- **Easy to use:** Bicep requires no previous knowledge of programming languages and is easy to write and understand.
- **Rich editor:** The editor provides rich type-safety, intellisense, and syntax validation, when you install the Bicep Extension for VS Code.

Azure Bicep files vs. ARM template

Bicep

```
param location string = resourceGroup().location
param storageAccountName string = 'toylaunch${uniqueString(resourceGroup().id)}'

resource storageAccount 'Microsoft.Storage/storageAccounts@2021-06-01' = {
  name: storageAccountName
  location: location
  sku: {
    name: 'Standard_LRS'
  }
  kind: 'StorageV2'
  properties: {
    accessTier: 'Hot'
  }
}
```

JSON

Copy

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "location": {
      "type": "string",
      "defaultValue": "[resourceGroup().location]"
    },
    "storageAccountName": {
      "type": "string",
      "defaultValue": "[format('toylaunch{0}', uniqueString(resourceGroup().id))]"
    }
  },
  "resources": [
    {
      "type": "Microsoft.Storage/storageAccounts",
      "apiVersion": "2021-06-01",
      "name": "[parameters('storageAccountName')]",
      "location": "[parameters('location')]",
      "sku": {
        "name": "Standard_LRS"
      },
      "kind": "StorageV2",
      "properties": {
        "accessTier": "Hot"
      }
    }
  ]
}
```

© Copyright Microsoft Corporation. All rights reserved.

The following are some benefits of Bicep:

- **Continuous full support** – Bicep provides support for all resource types and API versions for Azure services, which means that as soon as a resource provider introduces new resource types and API versions, you can use them in your Bicep file without waiting for a tool update.
- **Simple syntax** – Compared to an equivalent JSON file, Bicep files will be more concise and easier to read.
- **Easy to use**: Bicep requires no previous knowledge of programming languages and is easy to write and understand.
- **Rich editor**: The editor provides rich type-safety, intellisense, and syntax validation, when you install the Bicep Extension for VS Code.

Azure Bicep deployment

Azure PowerShell

```
New-AzResourceGroupDeployment -ResourceGroupName SampleRG -TemplateFile ./main.bicep  
-administratorLogin "<admin-login>"
```

Azure Command Line Interface (CLI)

```
az deployment group create --resource-group SampleRG --template-file -f `  
'\path\template.json'
```

© Copyright Microsoft Corporation. All rights reserved.

With PowerShell, save it as `main.bicep` on your local computer and run the commands above in PowerShell.

With Azure CLI, allows the same options for deployment scope as you have with PowerShell.

Azure CLI

Create Azure SQL Database

```
az sql server create --name ServerName --resource-group RGName `
--location Location --admin-user $login --admin-password $password
```

Create a firewall rule for Azure SQL Database

```
az sql server firewall-rule create --resource-group RGName `
--server ServerName -n AllowYourIp --start-ip-address 0.0.0.0 --end-ip-address
0.0.0.0
```

© Copyright Microsoft Corporation. All rights reserved.

The Azure Command-Line Interface (CLI) is a cross-platform command-line tool that helps you to create and manage Azure resources. You can run commands through the terminal using interactive command-line prompts or scripts.

Az.Sql PowerShell module

Returns information about an Azure SQL Database

```
Get-AzSqlServer -ResourceGroupName "ResourceGroup01" -ServerName "Server01"
```

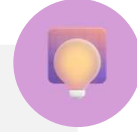
Create an Azure SQL Managed Instance database

```
New-AzSqlDatabase -ResourceGroupName "ResourceGroup01" -ServerName "Server01" -  
DatabaseName "Database01"
```

© Copyright Microsoft Corporation. All rights reserved.

Azure PowerShell and Azure CLI are both cross-platform command-line tools that will enable you to create and manage Azure resources on Windows, macOS and Linux. The main difference between the two is the shell environments that they support.

Knowledge check



What language are ARM templates written in?

- ☒ JSON
- ☐ C#
- ☐ T-SQL



If you want to pass in the region for a resource group deployment which option should you include in your template?

- ☒ Parameter
- ☐ Variable
- ☐ Output



Which element of a template allows for you to build dependencies into resources?

- ☒ dependsOn
- ☐ concat
- ☐ apiVersion

© Copyright Microsoft Corporation. All rights reserved.

1) What language are ARM templates written in?

- ☒ **JSON (korrekt):** ARM templates definieras i JSON-format för resurser och inställningar.
- ☒ **C# (fel):** C# kan användas för deployment, men inte ARM template-formatet.
- ☒ **T-SQL (fel):** T-SQL används för databaser, inte för Azure deployment templates.

2) If you want to pass in the region for a resource group deployment which option should you include in your template?

- ☒ **Parameter (korrekt):** Region anges som parameter så den kan varieras vid deployment.
- ☒ **Variable (fel):** Variabler beräknas internt och används inte för extern inmatning.
- ☒ **Output (fel):** Output används för att returnera värden efter deployment, inte input.

3) Which element of a template allows for you to build dependencies into resources?

- ☒ **dependsOn (korrekt):** dependsOn styr ordning och kräver att resurser skapas före andra.
- ☒ **concat (fel):** concat bygger strängar, men hanterar inga dependencies mellan resurser.
- ☒ **apiVersion (fel):** apiVersion anger resursens API-version, inte beroenden eller ordning.

Instructor led labs: Deploy an Azure SQL Database using an Azure Resource Manager template

**Explore Azure Resource
Manager template**

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/11-deploy-azure-database-using-template.html>

Create and manage SQL Agent Jobs

<https://learn.microsoft.com/training/modules/schedule-tasks-using-sql-server-agent/>

© Copyright Microsoft Corporation. All rights reserved.

Database systems need regular maintenance which includes tasks like making backups and updating statistics. Maintenance may also include regularly scheduled jobs that execute against a database. Some common examples of these jobs would be to extract, transform, and load data from a transaction processing system into a data warehouse. In SQL Server and Azure SQL Managed Instance, the SQL Server Agent service allows you to schedule jobs to perform these maintenance tasks (as well as providing other management functions).

Objectives



What Maintenance Activities you should perform on your databases



How to configure notifications and alerts on SQL Server Agent jobs and SQL Server



How to configure notifications alerts based on performance monitor values

© Copyright Microsoft Corporation. All rights reserved.

Database systems need regular maintenance which includes tasks like making backups and updating statistics. Maintenance may also include regularly scheduled jobs that execute against a database. Some common examples of these jobs would be to extract, transform, and load data from a transaction processing system into a data warehouse. In SQL Server and Azure SQL Managed Instance, the SQL Server Agent service allows you to schedule jobs to perform these maintenance tasks (as well as providing other management functions).

SQL Server maintenance activities

Databases need regular maintenance activities

Maintenance activities can be performed using maintenance plans or through open source tools

Some common examples of these activities include:

Database Backups

Database Consistency Checks

Index Maintenance

Statistics Updates

© Copyright Microsoft Corporation. All rights reserved.

You learned about the importance of backups and index/statistics maintenance in Learning Paths 4 and 5. Database consistency checks, also known as CHECKDB (for the command DBCC CHECKDB) are of equal importance, as it is the only way to check an entire database for corruption. Depending on the size of your databases and your uptime requirements, you may perform all of these activities nightly. More commonly in production systems, the maintenance operations are spread out over the course of week, as both index maintenance and consistency checks are very I/O intensive operations and they are typically done during weekend hours.

Maintenance plans create a workflow of the tasks to support your databases. Maintenance plans are created as Integration Services packages which allow you to schedule your maintenance activities. Many DBAs also use open source scripts to perform database maintenance, to allow for more flexibility and control of maintenance activities.

Best practices for maintenance plans

- Avoid using the database shrink task
- Stagger backups across servers and large databases to avoid potentially overwhelming your storage account or storage device
- Create separate maintenance plans for each set of maintenance activities

© Copyright Microsoft Corporation. All rights reserved.



Check Database Integrity

Check Database integrity on Local server connection
Databases: All databases
Include indexes
Physical only



Reorganize Index

Reorganize index on Local server connection
Databases: All databases
Object: Tables and views
Compact large object



Rebuild Index

Rebuild index on Local server connection
Databases: All databases
Object: Tables and views
Original amount of free space



Update Statistics

Update Statistics on Local server connection
Databases: All databases
Object: Tables and views
All existing statistics
Scan type: Full scan



Back Up Database (Full)

Backup Database on Local server connection
Databases: All databases
Type: Full
Append existing
Destination: Disk
Backup Compression (Default)

The best practice would be to create a maintenance plan for each type of task—and possibly even for a specific database on your server. For example, you might create a maintenance plan to backup system databases and another maintenance plan to backup user databases. You could also have another maintenance plan for special handling of the backup of one very large user database.

Demo: Building a maintenance plan

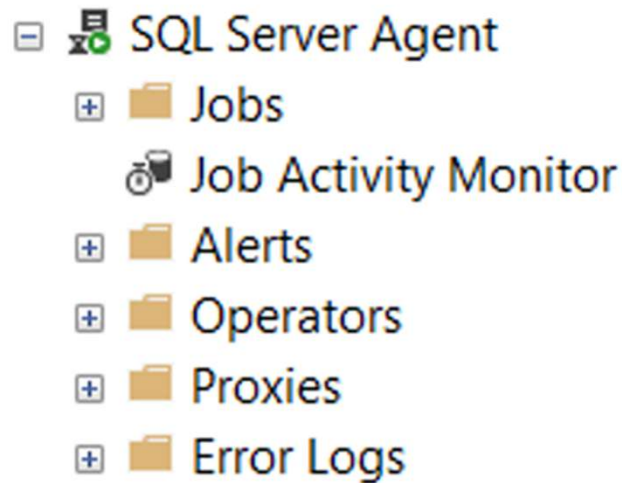
**Configuring backups and database
consistency checks**

© Copyright Microsoft Corporation. All rights reserved.

In this demo walk through the process of creating a maintenance plan in SQL Server Management Studio. Complete it, and ensure the agent jobs are scheduled. You can demo an execution if you have time.

SQL Server Agent

- Provides automation for SQL Server and Azure SQL Managed Instance
- Provides notifications for both job failures and alerts that are written to the error log
- The service can also notify administrators



© Copyright Microsoft Corporation. All rights reserved.

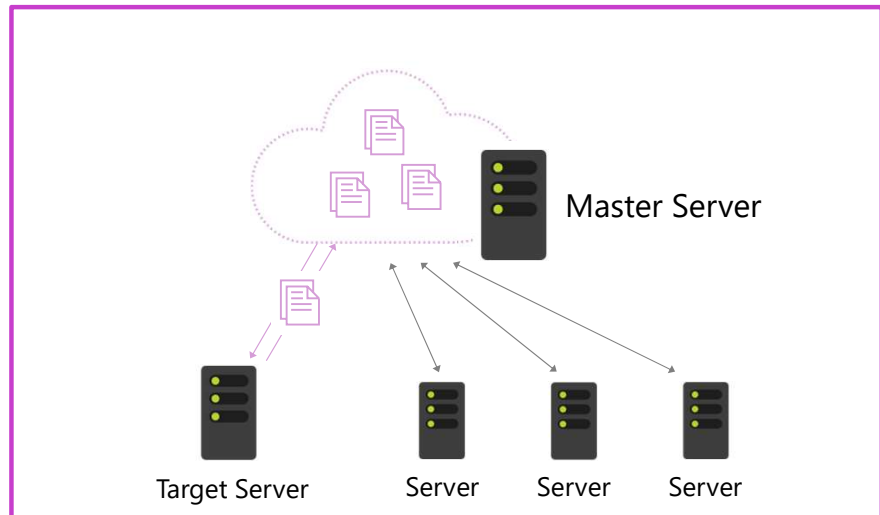
The SQL Server Agent is a process that runs alongside SQL Server, and controls job scheduling amongst other tasks.

SQL Server Agent uses SQL Server to store job information. Jobs contain one or more job steps. Each step contains its own task, for example, backing up a database.

SQL Server Agent can run a job on a schedule, in response to a specific event, or on demand. For example, if you want to back up all the company servers every weekday after hours, you can automate this task. Schedule the backup to run after 22:00 Monday through Friday; if the backup encounters a problem, SQL Server Agent can record the event and notify you.

Multi-server automation

- The SQL Server Agent supports designating one server as a primary to execute jobs on other target servers
- Target servers connect to the primary to ensure job schedules are updated on a regular basis
- This can be used to execute maintenance across your environment



© Copyright Microsoft Corporation. All rights reserved.

In a multi-server environment, the SQL Server Agent provides the option of designating one server as a master server that can execute jobs on other servers, designated as target servers. The master server stores a master copy of the jobs and distributes the jobs to the target servers. Target servers connect to the master server periodically to update their schedule of jobs. This allows you to define one job and deploy it across your enterprise. A good example of this would be configuring database maintenance across your environment. You could create a set of maintenance plan tasks just once and allow them to be pushed out to group of target servers, to ensure consistent deployment.

Task status notifications

The SQL Server Agent can be configured to notify an operator for job completion, success, and failure

Usually only enabled for failure to reduce notification volume

To deliver email notifications, the database mail and SQL Server Agent mail profile must be enabled

Can create alerts on performance conditions and error conditions found in the instance's error log

© Copyright Microsoft Corporation. All rights reserved.

Notification of completion is part of each SQL Server Agent job. You have the option of sending a notification on Job completion, failure, or success. Most DBAs notify on failure only, to avoid an influx of notifications for successful jobs

SQL Server Agent alerts allow you to be proactive with monitoring of your SQL Server. The agent reads the SQL Server error log and when it finds an error number for which an alert has been defined, it notifies an operator. In addition to monitoring the SQL Server errorlog, you can set up alerts to monitor SQL Server Performance conditions, as well as Windows Management Instrumentation (WMI) events. You can specify an alert to be raised in response to one or more events. A common pattern is to raise an alert on all SQL Server errors of level 16 and higher, and then add alerts for specific event types related to critical storage errors or Availability Group failover.

Another common use case for alerts is that DBAs may want to be notified in the event of certain server conditions. For example, if CPU utilization is over 90% for a period of five minutes, or Page Life Expectancy drops below a certain value. These conditions are based on the Windows Performance Monitor (perfmon) metrics that are tracked within the SQL Server database engine

Operators



Operators act as an alias for a user or group of users that have been configured in the SQL server agent



Typically, an operator will map to a group of administrators who will receive a notification or alert

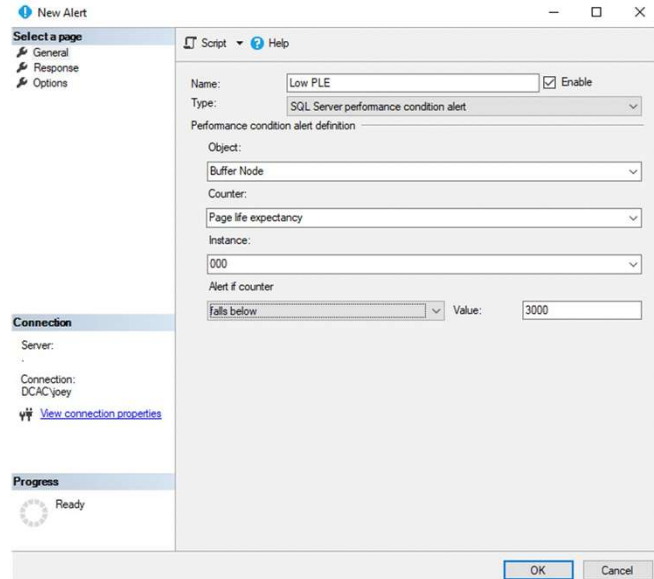
© Copyright Microsoft Corporation. All rights reserved.

Operators act as an alias for a user or group of users that have been configured to receive notifications of job completion, or to be informed of alerts that have been sent to the error log. An operator is defined as an operator name and contact information. Typically, an operator will map to a group of people using an email group. Having multiple people in the email group provides redundancy so that a notification is not missed if someone is unavailable. Groups are also beneficial if an employee leaves the organization; the single person can be removed from the email group and you do not have to update all of your instances.

Notifications based on metrics

Alerts are triggered in the event of certain server conditions:

For example, if CPU utilization is over 90% for a period of five minutes, or Page Life Expectancy drops below a certain value



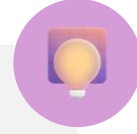
© Copyright Microsoft Corporation. All rights reserved.

SQL Server Agent alerts allow you to be proactive with monitoring of your SQL Server. The agent reads the SQL Server error log and when it finds an error number for which an alert has been defined, it notifies an operator. In addition to monitoring the SQL Server error log, you can set up alerts to monitor SQL Server Performance conditions, as well as Windows Management Instrumentation (WMI) events. You can specify an alert to be raised in response to one or more events. A common pattern is to raise an alert on all SQL Server errors of level 16 and higher, and then add alerts for specific event types related to critical storage errors or Availability Group failover. Another example would be to alert on performance conditions such as high CPU utilization or low Page Life Expectancy. You will learn how to implement alerts and notifications in the next section.

Notifications Based on Metrics

Another common use case for alerts is that DBAs may want to be notified in the event of certain server conditions. For example, if CPU utilization is over 90% for a period of five minutes, or Page Life Expectancy drops below a certain value. This is accomplished by creating performance condition alerts as shown in Figure 8. These conditions are based on the Windows Performance Monitor (perfmon) metrics that are tracked within the SQL Server database engine. You can reach the screen shown in here by right clicking on the SQL Server Agent in Object Explorer and choosing New > Alert

Knowledge check



What has to be configured before the SQL Server Agent can send e-mail?

- ☒ A mail profile
- ☐ An agent job
- ☐ An alert



Which system database stores SQL Server Agent jobs and their information?

- ☒ MSDB
- ☐ Master
- ☐ Model



Which operation recalculates the statistics on an index?

- ☒ Rebuild
- ☐ Reorganize
- ☐ Shrinking a file group

© Copyright Microsoft Corporation. All rights reserved.

1) What has to be configured before the SQL Server Agent can send e-mail?

- ☒ **A mail profile (korrekt):** Database Mail kräver mail profile innan Agent kan skicka e-post.
- ☒ **An agent job (fel):** Job kan trigga mail, men kräver ändå mail profile först.
- ☒ **An alert (fel):** Alerts kan skicka mail, men bara efter mail profile-konfiguration.

2) Which system database stores SQL Server Agent jobs and their information?

- ☒ **MSDB (korrekt):** SQL Server Agent sparar jobb, historik och metadata i MSDB.
- ☒ **Master (fel):** Master lagrar serverkonfiguration och logins, inte Agent-jobb.
- ☒ **Model (fel):** Model är mall för nya databaser, inte jobblagring.

3) Which operation recalculates the statistics on an index?

- ☒ **Rebuild (korrekt):** Index rebuild återskapar index och uppdaterar statistik automatiskt.
- ☒ **Reorganize (fel):** Reorganize defragmenterar bara; statistik uppdateras normalt inte.
- ☒ **Shrinking a file group (fel):** Shrink flyttar sidor och ökar fragmentering, inte statistik.

Instructor led labs: Create a CPU status alert for a SQL Server

Create an alert when a CPU exceeds an average of 80 percent

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/12-create-cpu-status-alert.html>

Manage Azure PaaS tasks by using automation

<https://learn.microsoft.com/training/modules/manage-azure-paas-resources-using-automated-methods/>

© Copyright Microsoft Corporation. All rights reserved.

You have learned about some of the capabilities of the SQL Server Agent. In this module you will learn about Azure Automation, Logic Apps, and Elastic Jobs, three approaches for automating jobs in PaaS. You will also learn to monitor automation tasks to make sure they're working properly.

Objectives



The benefits of Azure Policy



The capabilities of Azure Automation



How to use elastic jobs



How to use Logic Apps

© Copyright Microsoft Corporation. All rights reserved.

In this module you'll learn about Azure Automation, Logic Apps, and elastic jobs, three approaches for automating jobs in PaaS. You'll also learn about Azure policy and how it can be used to manage your subscription and costs.

Implementing Azure Policy



Azure Policy is used by administrators to ensure consistency across an Azure environment



Policy can be used to enforce naming conditions, limit the types of Azure resources deployed in a subscription, or limit the Azure regions where deployment is allowed



Multiple policies can be combined to create an initiative

© Copyright Microsoft Corporation. All rights reserved.

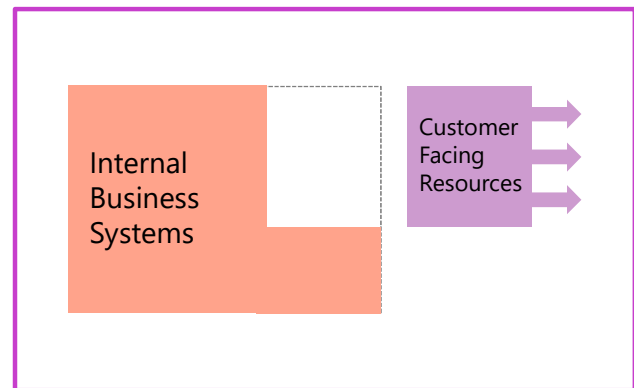
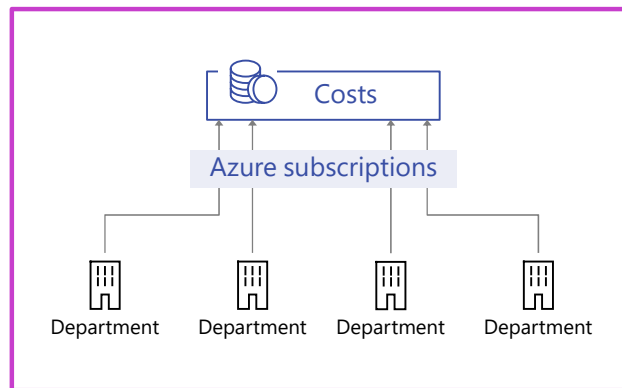
Group Policies or GPOs have been used by Windows server administrators for a long time, to manage security, provide consistency across the Windows Server environment in your organization. Some example of group policies include enforcement of password complexity, mapping shared network drives and configuring networked printers.

Azure offers similar features in ARM using Azure Policy. Policy provides a level of governance over your Azure subscriptions which enforce rules and controls over your Azure resources. Some examples of how you might use this include limiting the regions you can deploy a resource to, enforcing naming standards, or controlling resource sizes. Azure provides many example policies that you can use or you can define custom policies using JSON

Why would I have multiple Azure subscriptions?

Many large organizations use Azure subscriptions as a billing boundary so that costs are easily tracked back to a department

Other uses include separating customer facing resources from internal business systems



© Copyright Microsoft Corporation. All rights reserved.

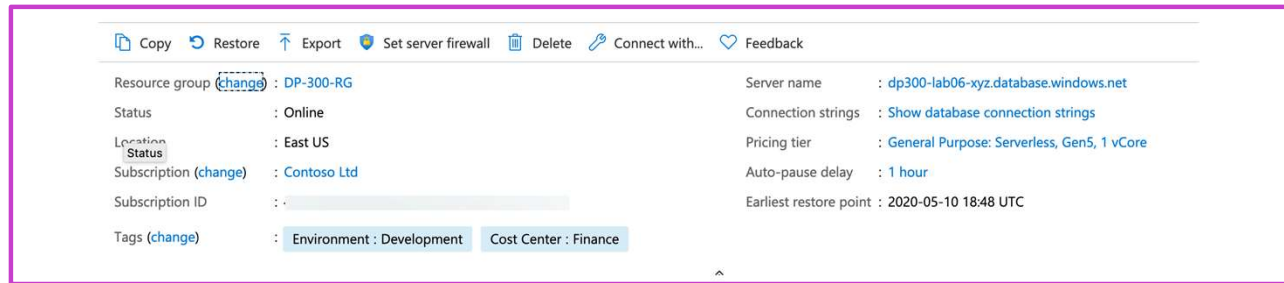
Organizations use multiple subscriptions for several reasons, including budget management, security, or isolation of resources. One example of this would be an organization that has both internal and customer facing resources. The internal resources could exist in one subscription, and the customer resources in another, for easier separation of billing and for isolation of the internal resources. These subscriptions may be managed together in a management group which allows you to manage policy and compliance across subscriptions.

Why should I tag Azure resources?

Tags allow you to describe your Azure resources beyond simply the resource name

Tags are key value pairs – Each Azure resource can have up to 15 tags

Some sample uses for tags include Cost Center, Environment, and CreateDate



© Copyright Microsoft Corporation. All rights reserved.

Tags are simply metadata that are used to better describe your Azure resources. These tags are stored as key:value pairs and appear in the Azure Portal associated with your Azure resources. Since they are associated with the resource, when you use PowerShell or Azure CLI commands, you can filter your commands based on tags.

Tags can be applied in the Azure Portal, or programmatically via PowerShell, Azure CLI, or as part of ARM template deployment. Tags can also be applied at the subscription, resource group, or individual resource level. Tags can also be modified at any time. Azure supports applying up to 15 tags to each Azure resource.

Azure Automation



Azure Automation allows you to create regularly scheduled tasks that run against any Azure resource or even on-premises virtual machines



Automation allows you to perform maintenance activities against Azure SQL Database



Automation can also be used to ensure consistent settings across multiple VMs

© Copyright Microsoft Corporation. All rights reserved.

For more complete control and granularity of your automation, Azure Automation allows for process automation, configuration management, full integration with Azure platform options (such as role-based access control and Microsoft Entra ID) and can manage Azure and on-premises resources.

Overview of Azure Automation components

Runbooks

Unit of execution in Azure Automation and may be created using PowerShell or Python. You have the option of deploying graphical runbooks, but they are limited in their capabilities

Modules

Used to execute PowerShell cmdlets within your runbooks. Load the modules for the PowerShell cmdlets you need for your runbooks

Credentials

Store sensitive information like passwords for use by runbooks

Schedules

Allow for runbooks to be scheduled for regular execution

© Copyright Microsoft Corporation. All rights reserved.

Runbooks - Runbooks are the unit of execution in Azure Automation. Runbooks can be defined as one of three types: a graphical runbook based on PowerShell, a PowerShell script, or Python script. PowerShell runbooks are most commonly used to manage Azure SQL resources.

Modules - Azure Automation defines an execution context for the PowerShell or Python code you are executing in your runbook. In order to execute your code, you need to import the supporting modules. For example, if you needed to run the *Get-AzSqlDatabase* PowerShell cmdlet, you would need to import the Az.SQL PowerShell module into your automation account.

Credentials - Credentials store sensitive information that runbooks or configurations can use at runtime.

Schedules - Schedules are linked to runbooks and trigger a runbook at a specific time.

Building an Azure Automation runbook



Building a runbook requires you to create automation account



For PowerShell runbooks, you will need to import the PowerShell modules you need in order to execute your runbook (e.g., Az.SQL)



You can also include credentials and run as accounts in your automation account



You can create schedules in your automation account and tie them to your runbooks



Automation includes a test pane you can test your code in the execution context of Azure Automation

© Copyright Microsoft Corporation. All rights reserved.

In order to build an automation runbook, you need to first create an automation account.

Demo: Configure an Automation runbook

PowerShell Runbook

© Copyright Microsoft Corporation. All rights reserved.

Create an automation account and demonstrate the process of loading a module and scheduling the runbook through simple steps.

Elastic Jobs in Azure SQL Database

-  Since Azure SQL Database lacks an agent, Elastic Jobs allow for a T-SQL scheduled execution methodology
-  Elastic Jobs require a dedicated SQL database to hold the metadata for your jobs
-  Define a target group, a SQL Database server, one or more databases or elastic pools as job targets
-  Database resources can run on different Azure subscriptions, and/or regions
-  Azure SQL Managed Instance doesn't support elastic jobs

© Copyright Microsoft Corporation. All rights reserved.

Elastic Jobs i Azure SQL Database är Microsofts sätt att ersätta **SQL Server Agent**, som saknas i Azure SQL Database. Tanken är att du ska kunna **schemalägga och köra T-SQL-jobb** mot en eller flera databaser, även om de ligger på olika servrar, elastic pools, prenumerationer eller regioner. För att använda Elastic Jobs behöver du en **dedikerad "job database"** som lagrar metadata: jobbdefinitioner, scheman, historik och status.

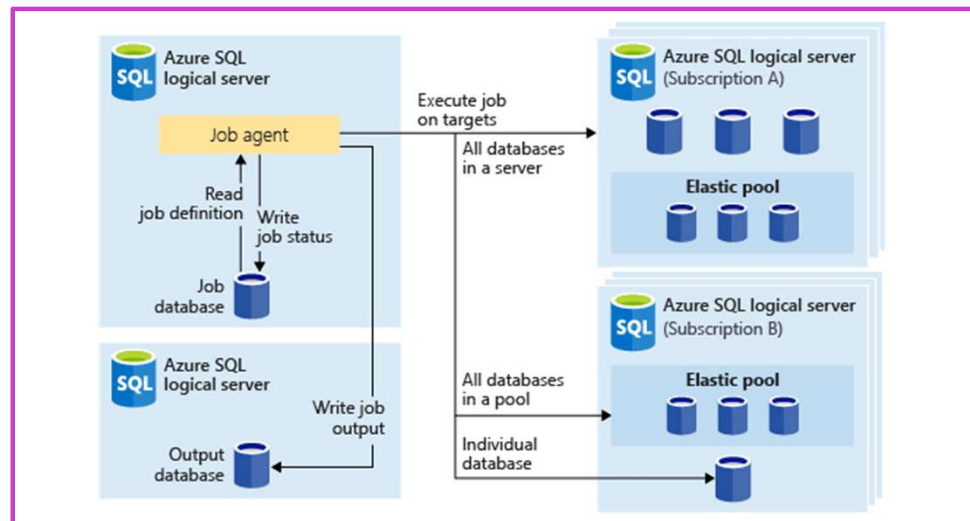
Därefter skapar du en **target group** där du definierar vilka resurser som ska påverkas, t.ex. en hel server, enskilda databaser eller pooler. Detta passar bra för administration i stor skala, exempelvis indexunderhåll, uppdatering av statistik, kontrollfrågor eller deployment-skript. Viktigt är att **Azure SQL Managed Instance inte stöder Elastic Jobs**, eftersom MI istället har mer klassisk Agent-funktionalitet.

One of the reasons why many DBAs became so familiar with Azure Automation is that Azure SQL Database initially lacked capabilities for scheduled jobs. The introduction of Elastic Jobs allows you to run a set of T-SQL scripts against a collection of servers or databases as a one-time job in an ad hoc manner, or by using a defined schedule. Elastic jobs work similarly to SQL Server Agent jobs, except that they are limited to executing T-SQL. The jobs work across all tiers of Azure SQL Database (excluding managed instance, which has a SQL Server Agent). To configure Elastic Jobs, you need a Job Agent and database dedicated to managing your jobs.

Elastic jobs

Components

- Elastic Job agent
- Job database
- Target group
- Job



© Copyright Microsoft Corporation. All rights reserved.

Elastic Jobs består av flera komponenter som tillsammans gör att du kan köra schemalagda T-SQL-jobb mot många Azure SQL-databaser samtidigt. **Elastic Job agenten** är själva "motorn" som startar jobb och håller koll på körningar. Den läser jobbdefinitioner från en särskild **Job database**, där all metadata lagras (jobb, schema, target groups, historik och status). En **Target group** definierar vilka databaser jobbet ska köras mot – det kan vara alla databaser på en server, alla databaser i en elastic pool eller enskilda databaser. När ett **Job** körs skickar agenten ut exekveringen till de valda målen och samlar in resultat. Utfallet kan även skrivas till en separat **Output database** för loggning och felsökning. Styrkan är att samma jobb kan köras centralt mot databaser i olika servrar och till och med olika subscriptions.

One of the reasons why many DBAs became so familiar with Azure Automation is that Azure SQL Database initially lacked capabilities for scheduled jobs.

Elastic jobs feature allows you to run a set of T-SQL scripts against a collection of servers or databases as a one-time job, or by using a defined schedule. Elastic jobs work similarly to SQL Server Agent jobs, except that they're limited to executing T-SQL. The jobs work across all tiers of Azure SQL Database. SQL Agent jobs continue to be used for task automation in SQL Server and are also included with Azure SQL Managed Instances.

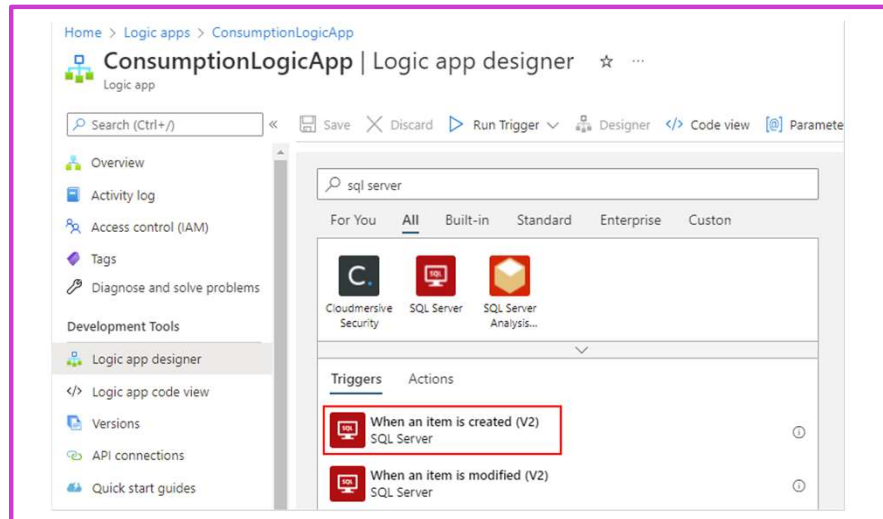
Let's review the elastic jobs components:

- **Elastic Job agent** - your Azure resource for running and managing jobs.
- **Job database** - a database dedicated to manage your jobs.
- **Target group** - a collection of servers, elastic pools, and single databases in which a job will be run.
- **Job** - one or more T-SQL scripts that compose a job step.

Azure Logic Apps

Platform for creating and running automated workflows that integrate apps, data, services, and systems

- Build your workflow from the components using the design tool.
- SQL Server connector supports the following SQL editions: **SQL Server, Azure SQL Database, and Azure SQL Managed Instance**
- Built-in SQL Server connector has **no triggers**
- Built-in SQL Server connector has only one operation: **Execute Query**



© Copyright Microsoft Corporation. All rights reserved.

Azure Logic Apps är en molntjänst i Azure för att skapa och köra **automatiserade arbetsflöden** (workflows) som kopplar ihop appar, data, tjänster och system – utan att du behöver skriva särskilt mycket kod. Man bygger flödet visuellt i en designer där man väljer **triggers** (vad som startar flödet) och **actions** (vad som ska göras).

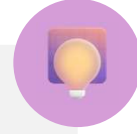
Ett vanligt användningsområde är integration mot databaser. Med **SQL Server-connectorn** kan Logic Apps ansluta till flera varianter av SQL: klassisk SQL Server, **Azure SQL Database** och **Azure SQL Managed Instance**.

På så sätt kan man t.ex. skapa ett flöde som reagerar på en händelse i ett system, hämtar data ur SQL och skickar vidare resultatet till exempelvis e-post, Teams, ServiceNow eller ett API.

Bilden visar också en viktig detalj: den **inbyggda SQL Server-connectorn** är begränsad. Den har **inga triggers** (dvs kan inte själv "lyssna" på ändringar i SQL) och erbjuder i princip bara en operation: **Execute Query**. Det innebär att SQL-delen ofta används som ett steg i ett större flöde, där triggeren kommer från något annat (t.ex. en timer, HTTP-request eller en fil som dyker upp). Logic Apps blir därmed ett smidigt verktyg för integration och automation kring SQL-miljöer.

Azure Logic Apps is a cloud-based platform for creating and running automated workflows that integrate your apps, data, services, and systems. With this platform, you can quickly develop highly scalable integration solutions for your enterprise and business-to-business (B2B) scenarios. As a member of Azure Integration Services, Azure Logic Apps simplifies the way that you connect legacy, modern, and cutting-edge systems across cloud, on premises, and hybrid environments.

Knowledge check



What is the unit of execution for your Azure Automation Account?

- ☒ Runbook
- ☐ Schedule
- ☐ Container



What scope can Azure Policy be deployed to?

- ☐ Tenant
- ☒ Subscription
- ☐ User



What is the name for the scope of SQL Elastic Job?

- ☒ Target Group
- ☐ Management Group
- ☐ Resource Group

© Copyright Microsoft Corporation. All rights reserved.

1) What is the unit of execution for your Azure Automation Account?

☒ **Runbook (korrekt):**

En runbook är själva skriptet som körs och utför automation.

☒ **Schedule (felaktigt):**

Ett schema triggar körning, men är inte själva exekveringsenheten.

☒ **Container (felaktigt):**

Containers kan köra kod, men Azure Automation använder runbooks.

2) What scope can Azure Policy be deployed to?

☒ **Tenant (felaktigt i detta sammanhang):**

Tenant-nivå används inte normalt som policy-scope i grundfrågan.

☒ **Subscription (korrekt):**

Policy kan tilldelas på prenumeration och gäller alla resurser där.

☒ **User (felaktigt):**

Azure Policy tilldelas inte på användarnivå, utan på resursscope.

3) What is the name for the scope of SQL Elastic Job?

☒ **Target Group (korrekt):**

Target group definierar vilka servrar/databaser/pooler jobbet körs mot.

☒ **Management Group (felaktigt):**

Management group gäller Azure governance, inte Elastic Jobs målurval.

☒ **Resource Group (felaktigt):**

Resource group är Azure-resurscontainer, inte Elastic Job scope.

Instructor led labs: Deploy an automation runbook to automatically rebuild indexes

Create an Automation account
Connect to an existing Azure SQL Database
Configure Automation Account assets
Create a PowerShell runbook
Create a schedule for a runbook

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/13-deploy-automation-runbook-rebuild-indexes.html>

References

Create an Azure Automation runbook

<https://learn.microsoft.com/azure/automation/automation-quickstart-create-runbook>

SQL Server Agent

<https://learn.microsoft.com/sql/ssms/agent/sql-server-agent?view=sql-server-ver15>

Ola Hallengren's Maintenance Solution

<https://ola.hallengren.com>

© Copyright Microsoft Corporation. All rights reserved.

Learning Path Recap

In this learning path, we:

Learned how to automate deployment of database resources. This involves understanding ARM template and Bicep structure, exploring deployment options for ARM templates, PowerShell, and Azure CLI, and using quick start Azure templates.

We also learned how to create and manage SQL Agent jobs. This includes learning the capabilities of the SQL Server Agent, configuring maintenance tasks for SQL Server, and creating notifications for failures and performance alerts.

Lastly, we explored how to manage Azure PaaS tasks using automation. This involves an introduction to Azure Automation, Elastic Jobs, and Azure Logic Apps, as well as creating an Azure Automation runbook.

© Copyright Microsoft Corporation. All rights reserved.



© Copyright Microsoft Corporation. All rights reserved.